Projeto de Análise de dados de Vendas

Integrantes do grupo:

- Amanda Tavares
- Enzo Ribeiro
- Gabryel da Silva
- Anthonio Cesar
- Cassio dos Santos
- Ana Heliza
- Rui Damião
- Nycolas Nathan

Nome da empresa: Pystock

Proposta: Utilizar 3 bibliotecas diferentes do Python para o cumprimento de uma complexa função: Analisar dados de vendas para que a empresa atendida tenha a melhor opção de dados possível e para a geração de um plano de ação que se adapte ao competitivo mercado empresarial.

Ou seja, nós fazemos filtros de dados, organização por colunas e transformamos tudo isso em gráficos acessados através de uma simples e intuitiva tela gráfica.

Objetivos:

- Análise de vendas através de gráficos.
- Interface gráfica simples e intuitiva.
- Utilização de filtros para organização dos dados.

Canva Com Apresentação:
https://www.canva.com/design/DAGiFILNXUo/c-BhRaL-TJa5lN-LUoOlNg/edit?utm_content=DAGiFILNXUo&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Programação explicada:

Arquivos .csv: Primeiramente, o nosso grupo fez três arquivos em formato de csv, usado para criar uma tabela de dados no aplicativo Visual Studio Code.

Arquivo Editar Seleção Ver Acessar Executar Terminal Ajuda

clientes.csv | produtos.csv | vendas.csv

vendas.csv > data

```csv
id_venda,id_produto,id_cliente,quantidade,preco_unitario,data_venda
1,101,1,3,20.00,2025-01-01
2,102,2,5,15.00,2025-01-02
3,103,1,2,25.00,2025-01-03
4,101,3,1,20.00,2025-01-04
5,102,2,4,15.00,2025-01-05
6,103,3,3,25.00,2025-01-06
7,101,4,6,20.00,2025-01-07
8,102,4,2,15.00,2025-01-08
9,104,5,10,35.00,2025-02-22
10,104,3,8,35.00,2025-02-23
```

Arquivo Editar Seleção Ver Acessar Executar Terminal Ajuda

clientes.csv | produtos.csv

produtos.csv > data

```csv
id_produto,nome_produto,categoria,regiao
101,Produto A,Esporte,Nordeste
102,Produto B,Roupas,Sul
103,Produto C,Comidas,Centro-Oeste
104,Produto D,Móveis,Sudeste
```

Arquivo Editar Seleção Ver Acessar Executar Terminal Ajuda

clientes.csv

clientes.csv > data

```csv
id_cliente, nome, data_nascimento, cpf
1, Rui, 1980-03-06, 11111111111
2, Anthonio, 1990-07-26, 22222222222
3, Amanda, 1983-02-08, 33333333333
4, Enzo, 1990-11-27, 44444444444
5, Ana, 1960-10-29, 55555555555
6, Cássio, 2015-04-20, 66666666666
7, Nycolas, 2001-09-11, 77777777777
8, Gabryel, 1990-08-03, 88888888888
9, Yuri Alberto, 2001-03-18, 99999999999
10, Larissa, 2010-10-26, 00000000000
```

Depois, com esses arquivos feitos, vamos usá-los para fazer uma transformação gráfica através de códigos, nas bibliotecas Matplotlib e Pandas dentro do Python.

Pandas:
Primeiro, vamos iniciar trazendo a biblioteca Pandas e os arquivos .csv para o código.

clientes.csv | produtos.csv | index.py | gleidson.py

index.py > ...

```python
import pandas as pd

# Função para carregar os dados
def carregar_dados():
    # Carrega os arquivos CSV
    produtos = pd.read_csv("produtos.csv")
    clientes = pd.read_csv("clientes.csv")
    vendas = pd.read_csv("vendas.csv")

```

Em seguida, removemos espaços em branco nas colunas e nos valores e Convertemos a coluna de data para o tipo datetime, o que facilita análises temporais.

```python
    # Remove espaços extras
    produtos.columns = produtos.columns.str.strip()
    clientes.columns = clientes.columns.str.strip()
    vendas.columns = vendas.columns.str.strip()

    # Remove espaços extras dos dados
    for df in [produtos, clientes, vendas]:
        for col in df.select_dtypes(include=["object"]).columns:
            df[col] = df[col].str.strip()

    # Converte data de venda para formato correto
    if "data_venda" in vendas.columns:
        vendas["data_venda"] = pd.to_datetime(vendas["data_venda"], errors="coerce")

    return produtos, clientes, vendas
```

Depois, criamos novas colunas, juntamos tabelas para podermos fazer uma melhor análise de dados e por fim, salvamos todas as informações dentro do dicionário.

```python
# Função para processar os dados
def processar_dados(produtos, clientes, vendas):
    # Cria a coluna de valor total
    vendas["valor_total"] = vendas["quantidade"] * vendas["preco_unitario"]

    # Junta os dados de vendas com os produtos
    vendas_produtos = vendas.merge(produtos, on="id_produto", how="left")

    # Vendas por produto
    vendas_por_produto = vendas_produtos.groupby("id_produto")["valor_total"].sum().reset_index()
    vendas_por_produto = vendas_por_produto.merge(produtos, on="id_produto", how="left")

    # Vendas por cliente
    vendas_por_cliente = vendas.groupby("id_cliente")["valor_total"].sum().reset_index()
    vendas_por_cliente = vendas_por_cliente.merge(clientes, on="id_cliente", how="left")

    # Vendas por categoria
    vendas_por_categoria = vendas_produtos.groupby("categoria")["valor_total"].sum().reset_index()

    # Vendas por região
    vendas_por_regiao = vendas_produtos.groupby("regiao")["valor_total"].sum().reset_index()

    return {
        "Vendas por Produto": vendas_por_produto,
        "Vendas por Cliente": vendas_por_cliente,
        "Vendas por Categoria": vendas_por_categoria,
        "Vendas por Região": vendas_por_regiao
    }

```

Finalmente, chamamos as funções anteriores e imprimimos os resultados das análises na tela.

```python
def main():
    produtos, clientes, vendas = carregar_dados()
    metricas = processar_dados(produtos, clientes, vendas)

    # Exibe as métricas
    for key, df in metricas.items():
        print(f"\n{key}:")
        print(df)
```

Resultado:

```
Vendas por Produto:
   id_produto  valor_total nome_produto categoria        regiao
0         101        200.0    Produto A   Esporte      Nordeste
1         102        165.0    Produto B    Roupas           Sul
2         103        125.0    Produto C   Comidas  Centro-Oeste
3         104        630.0    Produto D    M►veis       Sudeste

Vendas por Cliente:
   id_cliente  valor_total      nome data_nascimento          cpf
0           1        110.0       Rui      1980-03-06  11111111111
1           2        135.0  Anthonio      1990-07-26  22222222222
2           3        375.0    Amanda      1983-02-08  33333333333
3           4        150.0      Enzo      1990-11-27  44444444444
4           5        350.0       Ana      1960-10-29  55555555555

Vendas por Categoria:
  categoria  valor_total
0   Comidas        125.0
1   Esporte        200.0
2    M►veis        630.0
3    Roupas        165.0

Vendas por Regi►o:
        regiao  valor_total
0  Centro-Oeste        125.0
1      Nordeste        200.0
2       Sudeste        630.0
3           Sul        165.0
```

Matplotlib:

Para começarmos, iremos fazer todo o processo inicial que usamos no Pandas, porém, importamos agora o matplotlib.pyplot, que é usado para criar gráficos.

.

```python
import pandas as pd
import matplotlib.pyplot as plt

# Função para carregar os dados
def carregar_dados():
    produtos = pd.read_csv("produtos.csv")
    clientes = pd.read_csv("clientes.csv")
    vendas = pd.read_csv("vendas.csv")

    # Remove espaços extras
    produtos.columns = produtos.columns.str.strip()
    clientes.columns = clientes.columns.str.strip()
    vendas.columns = vendas.columns.str.strip()
```

Continuando, criamos um gráfico de linha que mostra a evolução das vendas ao longo do tempo.

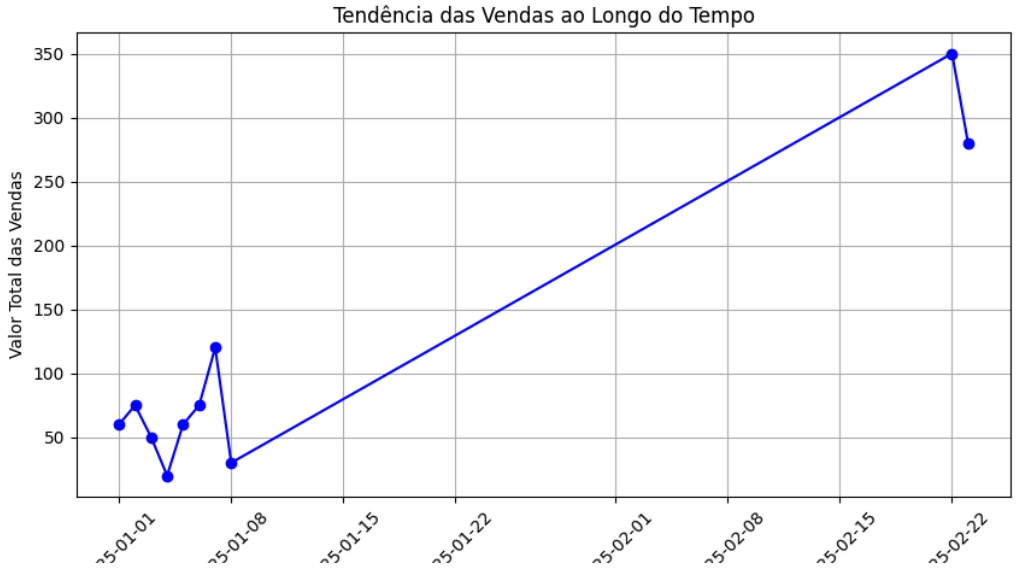
Ajustamos tamanho, cores, rótulos e exibimos a visualização.

```python
def gerar_graficos(vendas_por_data):
    plt.figure(figsize=(10, 5))
    plt.plot(vendas_por_data["data_venda"], vendas_por_data["valor_total"], marker="o", linestyle="-", color="b")
    plt.xlabel("Data")
    plt.ylabel("Valor Total das Vendas")
    plt.title("Tendência das Vendas ao Longo do Tempo")
    plt.xticks(rotation=45)
    plt.grid()
    plt.show()
```

Executamos todo o processo: carregamos os dados, processados por data e exibimos o gráfico.

```python
def main():
    produtos, clientes, vendas = carregar_dados()
    vendas_por_data = processar_dados(produtos, clientes, vendas)
    gerar_graficos(vendas_por_data)

if __name__ == "__main__":
    main()
```

Resultado:



Junção do Pandas e Matplotlib com o Tkinter:

Além das bibliotecas anteriores, iremos adicionar filedialog, que serve para abrir janelas para selecionar arquivos, messagebox, que exibe mensagens de alerta ou informação e FigureCanvasTkAgg, no qual permite inserir gráficos do matplotlib na janela do Tkinter.

```python
import tkinter as tk
from tkinter import filedialog, messagebox
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

Após isso, fizemos duas janelas para o usuário escolher os arquivos vendas.csv e produtos.csv. Depois, lemos os arquivos com pandas. Em seguida, usamos global para

permitir que outras funções usem as variáveis vendas e produtos.

Exibimos uma mensagem de sucesso com as primeiras linhas dos dados. Porém se der erro, adicionamos uma mensagem de erro com messagebox.

```
7   def carregar_arquivos():
8       try:
9           arquivo_vendas = filedialog.askopenfilename(title="vendas.csv", filetypes=
            [("CSV Files", "*.csv")])
10          arquivo_produtos = filedialog.askopenfilename(title="produtos.csv", filetypes=
            [("CSV Files", "*.csv")])
11
12          global vendas, produtos
13          vendas = pd.read_csv(arquivo_vendas)
14          produtos = pd.read_csv(arquivo_produtos)
15
16          messagebox.showinfo("Arquivos Carregados", f"Arquivos carregados com
            sucesso!\nPrimeiras linhas de Vendas:\n{vendas.head()}\n\nPrimeiras linhas de
            Produtos:\n{produtos.head()}")
17      except Exception as e:
18          messagebox.showerror("[ERRO]")
```

Nós também juntamos os dados de vendas com ou produtos, criamos a coluna “valor total”, somamos o valor total das vendas e criamos três gráficos.

```
19  def vendas_por_produto():
20      try:
21          vendas_produtos = vendas.merge(produtos, on="id_produto")
22          vendas_produtos["valor_total"] = vendas["quantidade"] * vendas
            ["preco_unitario"]
23          total_vendas_por_produto = vendas_produtos.groupby("id_produto")
            ["valor_total"].sum()
24
25          fig, ax = plt.subplots()
26          total_vendas_por_produto.plot(kind='bar', color='blue', ax=ax, title='Total
            de vendas por produto')
27          ax.set_xlabel('Produto')
28          ax.set_ylabel('Total de Vendas (R$)')
29          nova_janela = tk.Toplevel()
30          nova_janela.title("Gráfico de Vendas por Produto")
31          nova_janela.geometry("700x500")
32          canvas = FigureCanvasTkAgg(fig, master=nova_janela)
33          canvas.draw()
34          canvas.get_tk_widget().pack(side=tk.TOP, fill=tk.BOTH, expand=1)
35      except Exception as e:
36          messagebox.showerror("[ERRO]")
```

```python
def vendas_por_regiao():
    try:
        vendas_produtos = vendas.merge(produtos, on="id_produto")
        vendas_produtos["valor_total"] = vendas["quantidade"] * vendas["preco_unitario"]
        vendas_por_regiao = vendas_produtos.groupby("regiao")["valor_total"].sum()

        (variable) vendas_por_regiao: Series[Any]
        vendas_por_regiao.plot(kind='pie', autopct='%1.1f%%', ax=ax, title='Total de Vendas por Regiao')
        ax.set_ylabel('')
        nova_janela = tk.Toplevel()
        nova_janela.title("Gráfico de Vendas por Região")
        nova_janela.geometry("700x500")
        canvas = FigureCanvasTkAgg(fig, master=nova_janela)
        canvas.draw()
        canvas.get_tk_widget().pack(side=tk.TOP, fill=tk.BOTH, expand=1)
    except Exception as e:
        messagebox.showerror("[ERRO]")
```

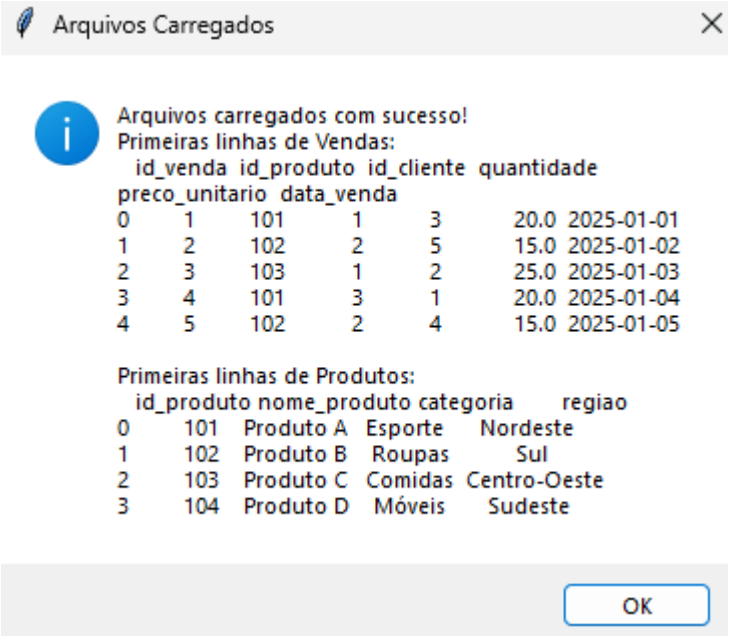
```python
def vendas_por_categoria():
    try:
        vendas_produtos = vendas.merge(produtos, on="id_produto", how="left")
        vendas_produtos["valor_total"] = vendas["quantidade"] * vendas
        ["preco_unitario"]
        vendas_por_categoria = vendas_produtos.groupby("categoria")["valor_total"].sum
        ()

        fig, ax = plt.subplots()
        vendas_por_categoria.plot(kind='bar', color='red', ax=ax, title='Total de
        vendas por Categoria')
        ax.set_xlabel('Produto')
        ax.set_ylabel('Total de Vendas (R$)')
        nova_janela = tk.Toplevel()
        nova_janela.title("Gráfico de Vendas por Categoria")
        nova_janela.geometry("700x900")
        canvas = FigureCanvasTkAgg(fig, master=nova_janela)
        canvas.draw()
        canvas.get_tk_widget().pack(side=tk.TOP, fill=tk.BOTH, expand=1)
    except Exception as e:
        messagebox.showerror("[ERRO]")
```

Agora, o nosso fez uma janela inicial, com os botões direcionados tanto para carregar arquivos, quanto para mostrar os gráficos.

```python
root = tk.Tk()
root.configure(bg='#F5DEB3')
root.title("Análise de Vendas")
root.geometry("800x600")

titulo = tk.Label(
    root,
    text="Avaliação de Dados e Vendas",
    font=("Arial", 40, "bold"),
    fg="#8B4513",
    bg=root.cget("bg"),
    pady=10
)
titulo.pack(pady=20)
btn_carregar = tk.Button(root, text="Carregar Arquivos", command=carregar_arquivos, font=("Arial", 20), bg='#8B4513', fg='#F5DEB3')
btn_carregar.pack(pady=20)

btn_vendas_produto = tk.Button(root, text="Gráfico de Vendas por Produto", command=vendas_por_produto, font=("Arial", 17), bg='#8B4513', fg='#F5DEB3')
btn_vendas_produto.pack(pady=20)
btn_vendas_regiao = tk.Button(root, text="Gráfico de Vendas por Regiao", command=vendas_por_regiao, font=("Arial", 17), bg='#8B4513', fg='#F5DEB3')
btn_vendas_regiao.pack(pady=20)
btn_vendas_categoria = tk.Button(root, text="Gráfico de Vendas por Categoria", command=vendas_por_categoria, font=("Arial", 17), bg='#8B4513', fg='#F5DEB3')
btn_vendas_categoria.pack(pady=20)

root.mainloop()
```

Resultados:

Análise de Vendas
Avaliação de Dados e Vendas
Carregar Arquivos
Gráfico de Vendas por Produto
Gráfico de Vendas por Regiao
Gráfico de Vendas por Categoria


Arquivos Carregados
Arquivos carregados com sucesso!
Primeiras linhas de Vendas:
id_venda id_produto id_cliente quantidade
preco_unitario data_venda
0 1 101 1 3 20.0 2025-01-01
1 2 102 2 5 15.0 2025-01-02
2 3 103 1 2 25.0 2025-01-03
3 4 101 3 1 20.0 2025-01-04
4 5 102 2 4 15.0 2025-01-05
Primeiras linhas de Produtos:
id_produto nome_produto categoria regiao
0 101 Produto A Esporte Nordeste
1 102 Produto B Roupas Sul
2 103 Produto C Comidas Centro-Oeste
3 104 Produto D Móveis Sudeste
OK

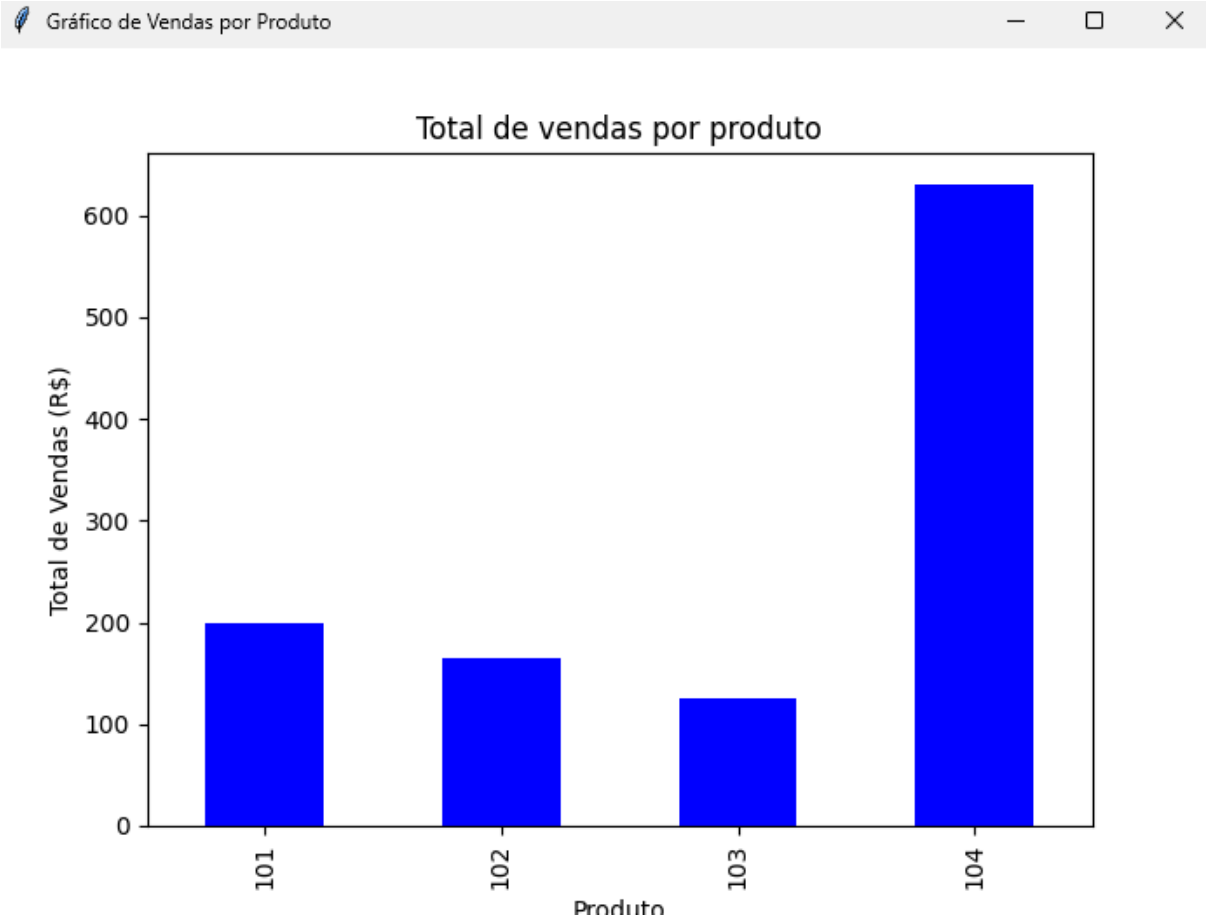


Gráfico de Vendas por Produto
Total de vendas por produto
Total de Vendas (R$)
0
100
200
300
400
500
600
101
102
103
104
Produto

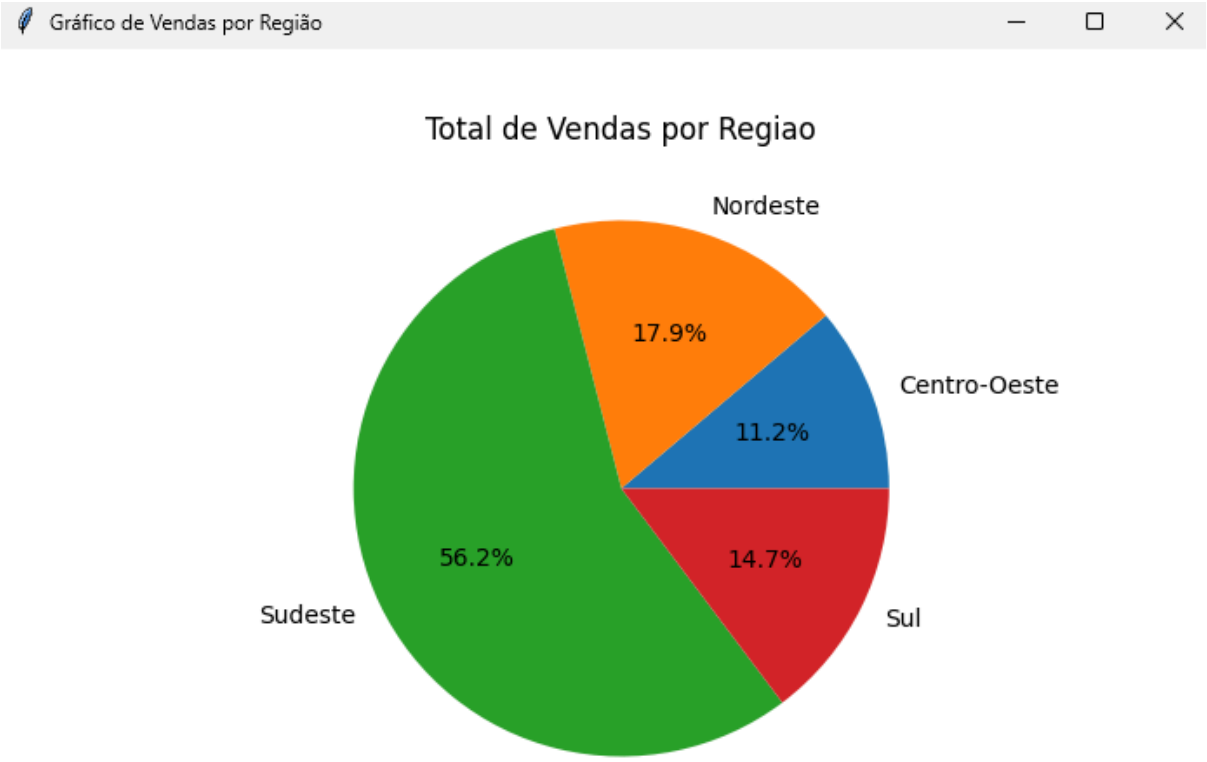
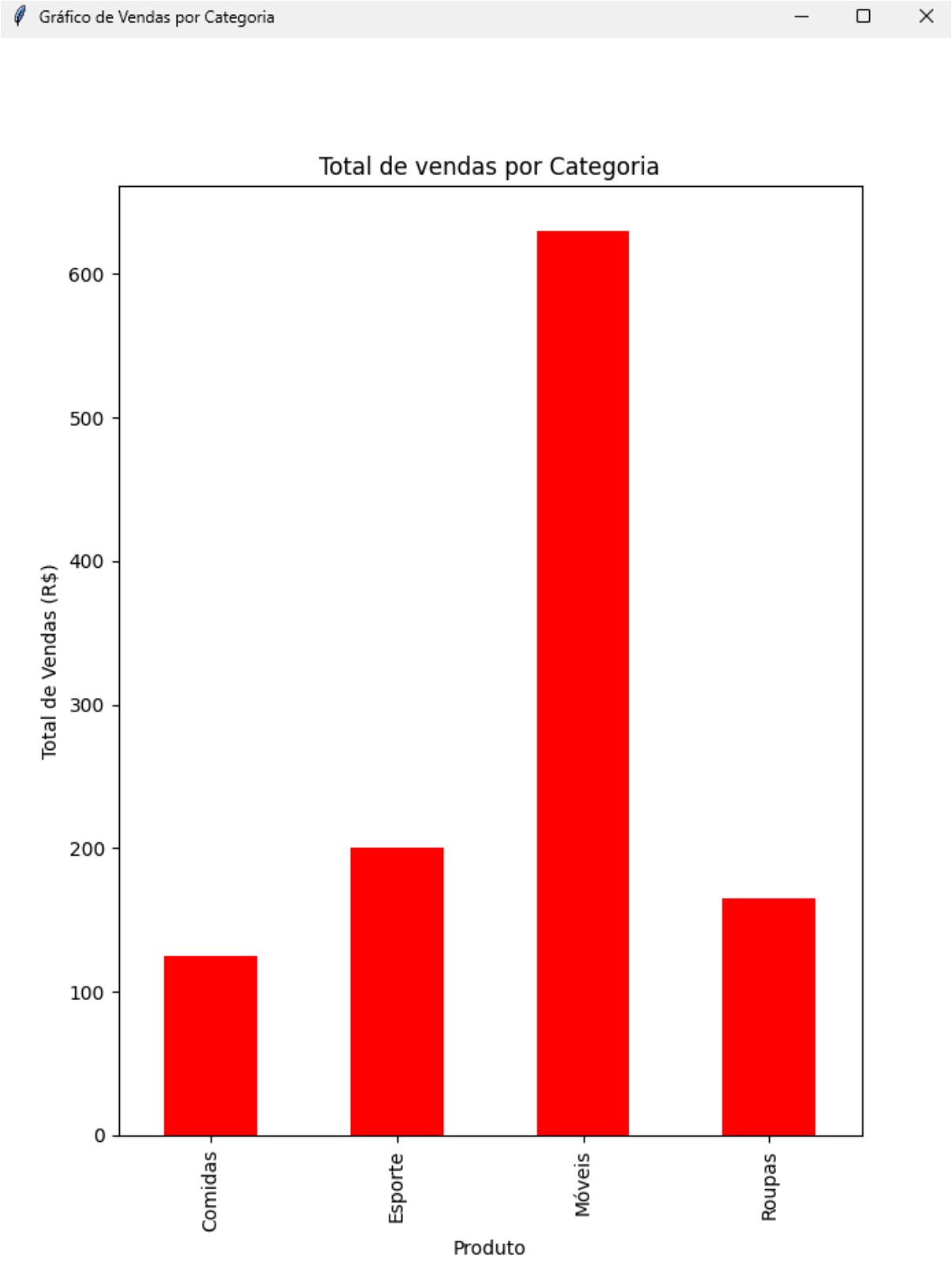


Gráfico de Vendas por Região
Total de Vendas por Regiao
Nordeste
17.9%
Centro-Oeste
11.2%
56.2%
14.7%
Sudeste
Sul



Conclusão do projeto:

Dado a programação, podemos perceber um sistema de análise de dados de vendas por gráfico de uma maneira mais organizada e sistemática para o entendimento dos funcionários ao analisar.